
Large Language Models think too much

Jeroen Niemendal (s3799042) Sebastian Jitaru (s6019595) Viktor Veselý (s3970043)

Abstract

Large Language Models (LLMs) often generate unnecessarily long reasoning chains, leading to high computational costs and inefficiency, even when shorter solutions would suffice. In this work, we propose a framework for reducing overthinking in LLMs while maintaining accuracy. A lightweight regressor predicts the minimal number of Chain-of-Thought (CoT) tokens required to solve a given problem based on the model’s initial latent state. We construct a preference dataset of minimal-length correct CoT sequences from multiple reasoning attempts. Empirical results on mathematical reasoning benchmarks demonstrate that our approach can reduce token length by more than half while slightly sacrificing accuracy relative to full CoT baselines and other fine-tuned models such as ThinkPrune. This work provides a practical method for adaptive and efficient reasoning in LLMs.

1. Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities in complex reasoning tasks, primarily due to techniques such as Chain-of-Thought (CoT) prompting (Wei et al., 2023). By generating intermediate reasoning steps, these models can decompose complex problems into smaller components, leading to more accurate and interpretable solutions. Recent advancements, such as the DeepSeek-R1 model trained with Group Reward Policy Optimization (GRPO), illustrate how Reinforcement Learning (RL) can effectively train models to produce extended CoT explanations (DeepSeek-AI et al., 2024).

Despite their success, such models incur significant computational costs. Systems trained to maximize correctness through rule-based rewards often exhibit excessive overthinking, producing unnecessarily long reasoning chains even when shorter, sufficient ones exist. This behavior increases inference latency and computational overhead, making large-scale deployment expensive. The core inefficiency lies in the reward mechanisms, which typically lack a means to regulate the conciseness of intermediate reasoning steps.

To address this issue, several research directions have been explored. One line of work introduces explicit length penalties into the RL objective (Team et al., 2025; Arora and Zanette, 2025; Dai et al., 2025b). For instance, ThinkPrune (Hou et al., 2025) applies an iterative pruning mechanism where the model is rewarded only if its answer is both correct and shorter than a target length that becomes increasingly restrictive over training. Although effective in reducing verbosity, this hard-constraint approach can be unstable. As demonstrated in (Dai et al., 2025a), strict penalties can cause training collapse, in which model accuracy sharply declines because certain problems genuinely require longer reasoning sequences to be solved correctly.

A more flexible strategy involves explicit length control. Length Controlled Policy Optimization (LCPO) (Aggarwal and Welleck, 2025) fine-tunes an LLM to follow a specified token count by appending a target length to the prompt. The reward function balances correctness with adherence to this target. While this formulation encourages output precision, its utility at test time is limited because the optimal target length for unseen problems is unknown.

Another direction focuses on adaptive reasoning using probing classifiers (Belinkov, 2022). In (Wang et al., 2025), a difficulty classifier trained on an LLM’s latent states and question context prompts the model to re-engage in reasoning when facing harder problems. Similarly, (Afzal et al., 2025) shows that a classifier can predict the final success of a CoT sequence from the model’s initial latent state, even before token generation begins. In principle, if success could be predicted early, the reasoning could be stopped sooner. However, this approach risks premature termination, where an almost-complete correct reasoning chain is truncated, leading to an incorrect answer (Afzal et al., 2025).

Taken together, these findings reveal a clear gap in the existing literature. Current methods are either too rigid, causing instability, dependent on unavailable information, such as the ideal target length at inference, or too coarse, relying on binary decisions that risk cutting off valid reasoning. An effective system should be both adaptive and controllable, it should

dynamically estimate the minimal sufficient reasoning length for each specific problem and then perform reasoning within that predicted target.

To achieve this, we propose a two-stage framework designed for dynamic and efficient reasoning. The system includes a lightweight regression module, referred to as the Think Regressor, trained on the LLM’s internal latent states to predict the minimal number of CoT tokens required for a correct solution. The second component is a finetuned LLM for shorter correct sequences, based on DeepScaleR-1.5B-Preview, fine-tuned to reliably generate reasoning sequences that adhere to a given token target. Instead of reinforcement learning, the model is optimized through Direct Preference Optimization (DPO) to prefer shorter correct reasoning chains over longer, redundant ones.

During inference, the user’s prompt is first processed by the LLM to extract its initial latent state. This representation is then passed to the Think Regressor, which predicts the minimum target tokens needed for a correct answer.

The main contributions of this work are threefold. First, we introduce the Think Regressor module, which predicts the required reasoning length directly from the LLM’s latent representation. Second, we present a methodology for constructing a preference dataset of minimal-length correct CoT sequences and fine-tuning a length-controllable model using DPO. Third, we provide an empirical evaluation demonstrating that our combined approach significantly reduces the average number of generated tokens while maintaining competitive accuracy relative to full CoT baselines.

2. Background

Chain-of-Thought (CoT) prompting enables LLMs to handle complex reasoning tasks by explicitly generating intermediate steps before producing a final answer (Wei et al., 2023). Instructing the model to reason step by step decomposes a problem into smaller, more tractable parts, improving performance in arithmetic, symbolic, and commonsense reasoning. However, this verbosity also contributes to high computational costs, motivating the need for efficient reasoning control.

Optimizing reasoning policies involves training methods that encourage models to generate high-quality reasoning trajectories. Group Reward Policy Optimization (GRPO), used in the DeepSeek-R1 model (DeepSeek-AI et al., 2024), extends standard policy gradient methods by evaluating multiple responses per prompt to construct a more robust reward signal and reference-based advantage function. Although effective in improving reasoning coherence, GRPO tends to produce long outputs. Length Controlled Policy Optimization (LCPO) (Aggarwal and Welleck, 2025) introduces a more explicit mechanism for regulating reasoning length by including deviations from a target token count in the reward function. By incorporating length sensitivity into the policy itself, LCPO allows the model to produce responses that approximate a desired output length.

Direct Preference Optimization (DPO) (Rafailov et al., 2024) offers a stable alternative to conventional reinforcement learning from human feedback (RLHF). Unlike RLHF, which requires training a separate reward model and applying policy gradient updates, DPO directly optimizes the model’s parameters using pairs of preferred and non-preferred responses. Each training example consists of a prompt and two responses, where the preferred response is explicitly favored through a loss function derived from the Bradley–Terry model. In our work, the preferred response corresponds to the shortest correct CoT sequence.

Finally, probing latent representations has emerged as a method to interpret what information neural networks encode internally (Belinkov, 2022). A probe is a small auxiliary model trained to infer specific properties, such as syntax, semantics, or task-related outcomes, from hidden activations. In LLMs, probing has been used to study the presence of linguistic and factual knowledge within intermediate layers. Recent work, such as (Afzal et al., 2025), demonstrates that the initial latent state of an LLM can predict the correctness of its final reasoning outcome. Building upon this insight, our Think Regressor extends the probing paradigm by replacing classification with regression.

3. Methodology

The following models are used for training or comparison:

- The L1-Exact, from the LCPO family (Aggarwal and Welleck, 2025), model for dataset generation and regressor test evaluation (explained better in Section 3.1);
- The base model DeepScaleR-1.5B-Preview (Luo et al., 2025) was used to train L1. We also finetune the base model using the think-reducing DPO method;

- ThinkPrune (Hou et al., 2025), which is a finetuned version of the base model for thinking reduction.

Two approaches are developed to shorten the unnecessarily long reasoning, pushing the model towards shorter and correct answers using DPO and predicting (using an external MLP) the minimal number of tokens that are needed for successfully answering the prompt.

The methods (DPO, regressor, and ThinkPrune) are evaluated based on how much they can reduce the number of thinking tokens whilst maintaining the accuracy of the baseline model. All the approaches are evaluated on the same test dataset with the temperature set to zero.

3.1. Dataset

The experiments use two mathematical reasoning datasets: the MATH dataset (Hendrycks et al., 2021) and the American Invitational Mathematics Examination (AIME) dataset (Veeraboina, 2023). The MATH dataset contains 12,500 problems, each annotated with a natural language problem statement, a full solution whose final answer is boxed, a difficulty level from 1 to 5, and a categorical topic label. The AIME dataset contains a total of 933 problems alongside their solution.

To study the relationship between reasoning length and correctness, we generate multiple solution attempts per problem using the L1-Exact model (Aggarwal and Welleck, 2025) (denoted L1). By building the prompts, concatenating the problems with an instruction specifying a target reasoning length of the form: “Let’s think step by step inside and output the final answer within boxed{ }. Think for [target_tokens] tokens.” Targets are linearly spaced between 100 and 2500 tokens, $\mathcal{T} = 100, 366, \dots, 2500$. For every generated sample, we record the full output, the text within the reasoning tags, the target and realized token counts, and the correctness of the prediction is determined via a string comparison against the reference solution. We also extract the model’s last token hidden state h before generation.

Preliminary analysis of these generated samples revealed that many low-difficulty MATH problems (Levels 1–2) are solved successfully by L1 even at very small token targets. Figure 1 shows that the minimum required reasoning length is skewed toward short traces for such items. Because these problems provide little information about how solution quality varies with reasoning length and would bias regression models toward trivial short reasoning, we construct a filtered subset of 3829 MATH problems emphasizing higher difficulty. Specifically, we retain all Level 3–5 problems and include Level 0–2 problems only up to problem ID 1888. This cutoff balances difficulty and compute constraints: generating ~ 2000 problems across 10 targets requires approximately 26 hours on a single NVIDIA A100 (batch size 64), making full-dataset scaling infeasible.

Additionally, we include the first 660 AIME problems. The AIME questions are generally harder than the MATH dataset questions, requiring more tokens, as can be seen in the right side of Figure 1. The reason behind not using the full dataset is time and computing constraints. In the merged dataset, AIME problems are assigned a synthetic Level 6 label.

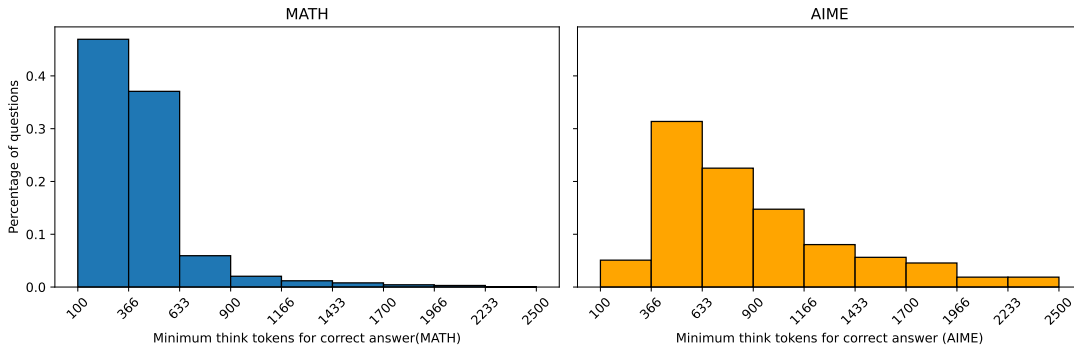


Figure 1. Distribution of minimum required tokens to answer question correctly per dataset

From the generated multi-target samples, we construct two learning datasets:

- A multi-label classification dataset with targets indicating correctness at each target token
- A Direct Preference Optimization dataset where the “chosen” sample is the shortest-target correct solution and the

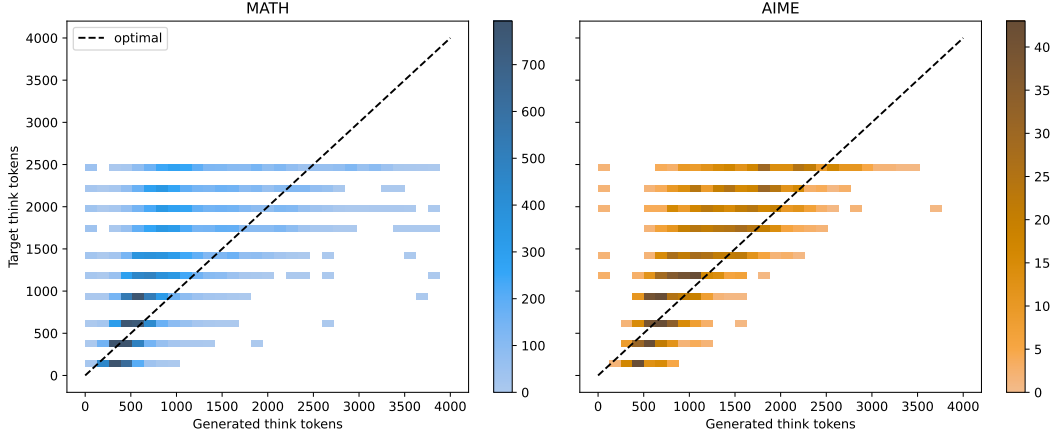


Figure 2. Adherence of the model to the desired number of target tokens. (Left) MATH dataset (Right) AIME dataset

“rejected” sample is the longest-target solution. To prevent trivial cues, explicit token-target instructions are removed from DPO prompts.

To ensure evaluation on unseen reasoning trajectories, we construct a stratified test set containing 20 problems per difficulty level (Levels 1–5 from MATH and Level 6 from AIME), totaling 120 unique questions. If a question is included in the test set, all 10 of its generated solutions are withheld from training. The training set, therefore, contains 4369 unique questions, while the full dataset spans 4489 unique problems.

3.2. DPO

The base model (DeepScaleR-1.5B-Preview) is fine-tuned using DPO on a dataset on a filtered trainset. First, all answers with Chinese characters were filtered out. Then, for each prompt, a response pair was created. Each pair consists of the shortest correct answer and the longest answer. Then, using DPO the model was finetuned to prefer the shortest correct answer and disprefer the longest answer. If there was no correct answer given for a prompt, or only the longest answer was correct, then this prompt was not used to train the model.

DeepScaleR-1.5B-Preview was loaded with half-precision (float16) for memory efficiency. A LoRA configuration was applied with a rank of 8, an alpha of 16, and a dropout of 0.05. DPO training was configured with a learning rate of $1e-5$. Training was performed for 2 epochs.

3.3. Think regressor

The regressor takes as the input the hidden state of the last token $h \in \mathbb{R}^{1536}$ after inputting the problem statement but before adding the “Let’s think step by ...” phrase. The output of the regressor should be the “minimal” number of CoT tokens required for correctly answering the problem. Recall that for each problem, ten CoT trajectories of different lengths are generated. The naive approach would only pick the shortest correct trajectory (for each question) and train the regressor on those, discarding the other trajectories. To enrich our learning signal and utilize all the data, the target tokens were discretized using ten bins (with values [100, 366, 633, 900, 1166, 1433, 1700, 1966, 2233, 2500]). The bins corresponded to the ten different trajectories’ lengths, however, in future research, other binning methods could be used. Thus, the regressor predicts a binary outcome for each bin, whether for that trajectory length, the answer will be successful. Formally, regressor f is a function $f : \mathbb{R}^{1536} \rightarrow \{0, 1\}^{10}$.

The regressor is initialized as an MLP with the following layers [1536, 1024, 512, 256, 10], where each linear layer, except the last one, is followed by a ReLU activation function. The sigmoid function is used for the last layer. Weight decay of $1e-4$ with 20% dropout is used to regularize the regressor. Other architectures were tested but yielded no validation performance benefits.

The 4.3K non-test data is further split to 85% training and 15% validation. Adam optimizer with $1e-4$ learning rate is used

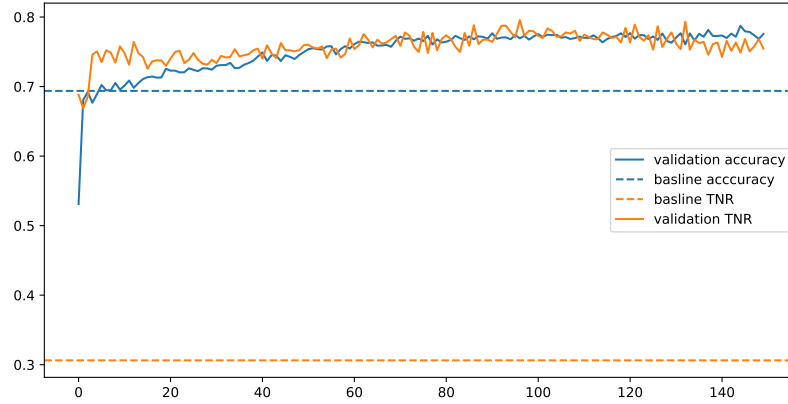


Figure 3. Validation accuracy and TNR over the think regressor training epochs. Baseline metrics are calculated using the training data label distribution.

	Accuracy		Average Tokens Generated	
	MATH100	AIME20	MATH100	AIME20
DeepScaleR-1.5B-Preview	82	45	2316	5370
ThinkPrune	82	45	1875	4070
DPO	80	30	2270	5479
Static Regressor	74	25	738	1968
Dyanmic Regressor	33	15	1697	1422

Table 1. Comparison of model performance and average tokens generated.

to train the model for 150 epochs. Due to unbalanced labels (30% False; 70% True) weighted binary cross entropy (with double negative weighting) is used for training. During training, we keep track of validation accuracy and true-negative rate (TNR). Baseline accuracy and TNR are calculated using the training data label distribution. Figure 3 shows the validation metrics over the training epochs.

To pick the actual CoT token target for the L1 model, the last token hidden state is acquired from L1 h , the regressor predicts each bin’s correctness $f(h)$, and the minimal correct bin is chosen. If no correct bin is present $1 \notin f(h)$, then the bin with the highest corresponding regressor logit is picked.

During the test inference evaluation, we use a regressor in two modes *static* and *dynamic*. In *static* mode, the token target is evaluated only once after inputting the problem statement. In *dynamic* mode, the token target is re-evaluated every 50 tokens using the new hidden state $f(h_{i:50})$ (for every i until the end of generation) and the new target is overwritten (not appended) in the prompt. To fully reflect the retrospective prompt changes, the KV-cache is cleared.

4. Results

In Table 1, it can be seen that the static regressor performs worse in terms of accuracy on the MATH and AIME test datasets, compared to the DeepScaleR and ThinkPrune models, but is able to reduce the tokens produced by more than half. The DPO method performs worse in terms of accuracy than the non-fine-tuned model while producing a similar number of tokens. The dynamic regressor performed by far the worst in terms of accuracy, while requiring more tokens than the static regressor.

As can be seen in Figure 4, the static regressor performs similarly or better than the other methods in terms of accuracy for level 1, 2, 3, and 4 questions, with less than half the number of tokens used. For the harder questions, level 5 and level 6, the regressor method does get outperformed in terms of accuracy.

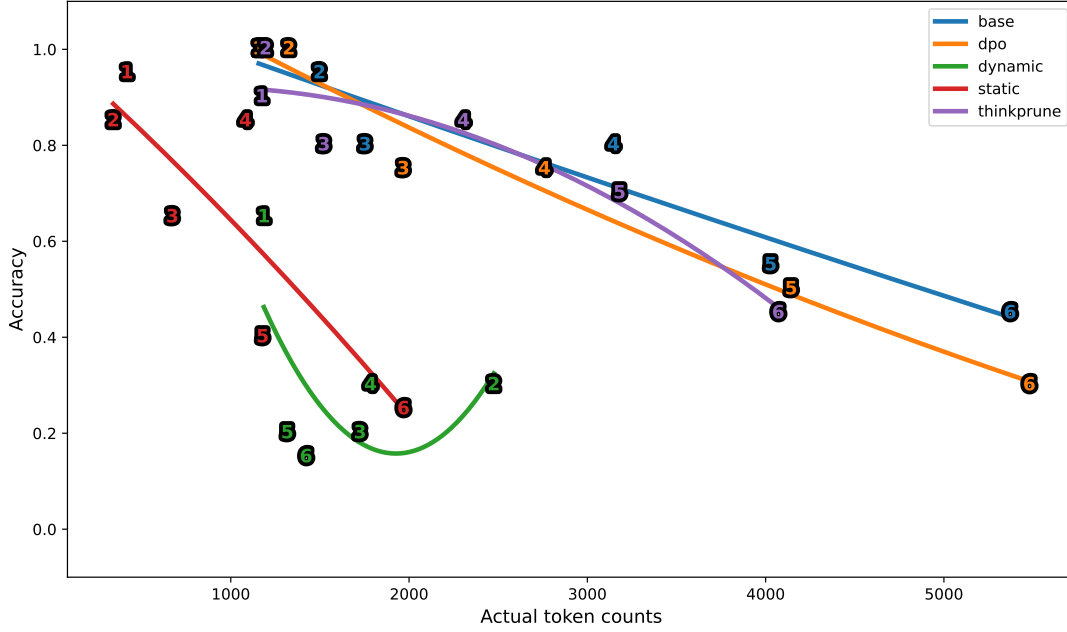


Figure 4. The test inference results, actual generated token counts against the accuracy. The results were grouped by the question level difficulty, and the levels are displayed as the scatter points. The following methods are compared: base (no training) (Dai et al., 2025a), dpo (fine-tuned base), dynamic (regressor), static (regressor), and thinkprune (Hou et al., 2025)

5. Discussion

5.1. DPO

The DPO model did not reduce the number of tokens produced and decreased the accuracy compared to the baseline. This is a rather unexpected result since (Arora and Zanette, 2025) showed that it is possible to reduce the number of tokens using DPO.

We theorize that the reason for the model performing worse is due to the dataset it was trained on. In Appendix A can an example be seen of the L1 model hallucinating. We noticed that this happened quite often, which might explain why the fine-tuned model using the L1 answers underperformed. In future research, the answers should either be cleaned by detecting when the answers contain many duplicate lines, or the answers should be manually verified.

5.2. Static Regressor

As discussed in the results, the regressor method gets outperformed in terms of accuracy by the baseline methods for the level 5 and AIME questions. One thing to note is that the ThinkPrune model was fine-tuned on the AIME dataset, which therefore is not a fair comparison.

Post analysis also shows that for eight of the 100 questions in the MATH test dataset, the L1 model started hallucinating, resulting in the model not giving a final answer. The base model and the ThinkPrune model answered these questions correctly.

Another possible explanation for the discrepancy in accuracy is that the regressor is too conservative in the amount of tokens it predicts. This could be resolved by creating bins with higher target token lengths. As it is very likely that this would have improved the results for the level 5 and AIME question.

A clear trend can be seen in Figure 4, where the amount of tokens increases for each increase in difficulty level. We theorize that if the slope of the predicted tokens per difficulty were closer to that of the base model, it would have performed just as well or better. While still being able to reduce the number of tokens generated by half, compared to the base model. One way to get closer to this trend would be by creating a dataset that generates longer answers for each question. The upper

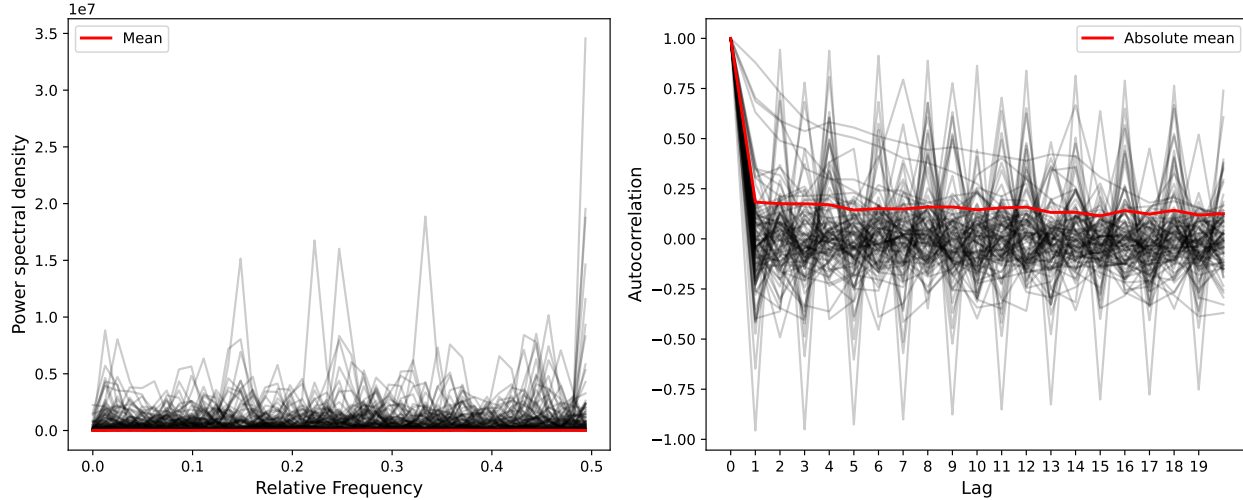


Figure 5. Analysis of target tokens’ temporal patterns with dynamic regressor. On the left: power spectral analysis of the trajectories with relative frequencies. On the right: autocorrelation analysis of the trajectories, where the absolute mean is shown in red.

limit of the 2500 tokens may not be high enough.

Ideally, the training set size would also be increased for the regressor. We found that the regressor would always output true for every bin if it was trained with considerably more correctly answered questions than incorrectly answered questions. For that reason, we chose, as previously mentioned, a 30-70 incorrect-correct split. However, due to the relatively easy questions of the MATH dataset, compared to the AIME dataset, most of the questions were answered correctly. Which meant we were left with a rather small dataset. In future works, the dataset size can be increased by including more difficult questions, by expanding the AIME dataset size, but also by adding other more difficult benchmarks like the Olympiad (He et al., 2024).

In future research, the test dataset size should also be increased. Now it is not possible to make any claims about statistically significant results in terms of the accuracy of the different models. Moreover, to ensure a fair comparison with the ThinkPrune model, a test dataset other than AIME should be used.

5.3. How does the dynamic regressor update its guess?

We claim that, for most thinking trajectories with the dynamic regressor, the variations in target tokens show no clear pattern and behave like white noise (not generalizing properly). Since plotting the raw targets over time yields an uninformative signal, Figure 5 instead presents a power spectrum analysis (left) and an autocorrelation plot (right). The individual 120 test target tokens’ trajectories are plotted in black color and their mean is plotted with a red color.

Recall that the frequency spectrum of white noise is uniformly spread over all the frequencies. The mean power spectral density (red line, left plot) also has a constant value over all the relative frequencies. The autocorrelation plot also indicates that the majority of the target trajectories exhibit almost no pattern since the absolute mean is below 0.25.

However, there exist a few target trajectories for which the guess was oscillating up and down (at each step). This can be seen by a few trajectories having a spike of power density at 0.5 relative frequency and by the autocorrelation oscillating between -1 and 1.

Furthermore, there exist even fewer target trajectories (around 4) for which the autocorrelation is decreasing slowly from a value 1. This could occur when the trajectories are constant or change linearly (causing correlation to drop due to padding), or when they exhibit mild nonlinear trends such as exponential or quadratic growth or decay.

5.4. Conclusion

This work takes a step toward reducing redundant reasoning in large language models by predicting and constraining the minimal thinking length required for accurate problem solving. The proposed combination of the static regressor shows

that adaptive control can substantially cut reasoning length without catastrophic accuracy loss. However, the approach remains limited by dataset scale, regressor calibration, and the simplicity of the current binning strategy. Future work should focus on improving the dynamic regressor, either by training it directly on mid-reasoning hidden states to capture evolving problem difficulty, or by simplifying its architecture to a near-linear form to increase generalization. Ultimately, improving models' awareness of when to stop thinking may prove as important as teaching them how to think.

References

- Anum Afzal, Florian Matthes, Gal Chechik, and Yftah Ziser. Knowing before saying: Llm representations encode information about chain-of-thought success before completion. *arXiv preprint arXiv:2505.24362*, 2025.
- Pranjal Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*, 2025.
- Daman Arora and Andrea Zanette. Training language models to reason efficiently. *arXiv preprint arXiv:2502.04463*, 2025.
- Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022.
- Muzhi Dai, Shixuan Liu, and Qingyi Si. Stable reinforcement learning for efficient reasoning. *arXiv preprint arXiv:2505.18086*, 2025a.
- Muzhi Dai, Chenxu Yang, and Qingyi Si. S-grpo: Early exit via reinforcement learning in reasoning models. *arXiv preprint arXiv:2505.07686*, 2025b.
- DeepSeek-AI, Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y. Wu, Yukun Li, Huazuo Gao, Shirong Ma, Wangding Zeng, Xiao Bi, Zihui Gu, Hanwei Xu, Damai Dai, Kai Dong, Liyue Zhang, Yishi Piao, Zhibin Gou, Zhenda Xie, Zhewen Hao, Bingxuan Wang, Junxiao Song, Deli Chen, Xin Xie, Kang Guan, Yuxiang You, Aixin Liu, Qiushi Du, Wenjun Gao, Xuan Lu, Qinyu Chen, Yaohui Wang, Chengqi Deng, Jiashi Li, Chenggang Zhao, Chong Ruan, Fuli Luo, and Wenfeng Liang. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence, 2024. URL <https://arxiv.org/abs/2406.11931>.
- Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. *arXiv preprint arXiv:2402.14008*, 2024. URL <https://arxiv.org/abs/2402.14008>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *NeurIPS*, 2021.
- Bairu Hou, Yang Zhang, Jiabao Ji, Yujian Liu, Kaizhi Qian, Jacob Andreas, and Shiyu Chang. Thinkprune: Pruning long chain-of-thought of llms via reinforcement learning. *arXiv preprint arXiv:2504.01296*, 2025.
- Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. <https://pretty-radio-b75.notion.site/DeepScaler-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303>, 2025. Notion Blog.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2024. URL <https://arxiv.org/abs/2305.18290>.
- Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.
- Hemish Veeraboina. Aime problem set 1983-2024, 2023. URL <https://www.kaggle.com/datasets/hemishveeraboina/aime-problem-set-1983-2024>.
- Jianwei Wang, Ziming Wu, Fuming Lai, Shaobing Lian, and Ziqian Zeng. Synadapt: Learning adaptive reasoning in large language models via synthetic continuous chain-of-thought. *arXiv preprint arXiv:2508.00574*, 2025.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL <https://arxiv.org/abs/2201.11903>.

[illegible]